

Credit Card Fraud Detection

Using Anomaly Detection Techniques | Academic project | Version 1.0

PRODUCT OVERVIEW	A web-based monitoring system that scores credit-card transactions for unusual patterns and helps analysts review potential fraud. The project demonstrates anomaly detection on highly imbalanced transaction data; its alerts support investigation and do not make final financial decisions.
------------------	--

1. Problem & objectives

Fraud is rare, diverse, and can shift over time, making fixed rules and accuracy-only evaluation inadequate. The product will surface transactions that differ from learned normal behavior, provide interpretable risk signals, and let users review outcomes in a single dashboard.

2. Objectives

OBJECTIVES	
Detect unusual transactions	Compare three anomaly detection methods on the same prepared dataset.
Prioritize review	Return a normalized anomaly score and flag for each transaction.
Enable monitoring	Expose predictions and summary metrics through an API and dashboard.
Support evaluation	Report precision, recall, F1 and PR-AUC when labeled ground truth is available.

3. Target users

Primary: fraud analysts or project evaluators reviewing flagged transactions and model results. Secondary: a system administrator who loads data, selects a model, and monitors service health.

4. Core features

Transaction ingestion and validation; anomaly scoring and configurable alert threshold; searchable transaction list with risk score and model flag; transaction detail view; dashboard summaries for volume, flagged share, score distribution and evaluation metrics; model comparison for the three supported algorithms.

How the system works

MVP requirements for training, scoring, review and evaluation

5. Machine-learning approach

Train and compare Isolation Forest, Local Outlier Factor (novelty mode for scoring unseen records), and One-Class SVM on scaled numeric features. Fit preprocessing only on training data; use a held-out set for evaluation. Convert model decision values to a consistent anomaly score and flag using a documented threshold. Use class-aware metrics (precision, recall, F1, PR-AUC) where labels exist; otherwise report review-oriented score distributions and acknowledge that unsupervised flags are not confirmed fraud.

6. Technology stack

Python for model and API; Pandas and NumPy for data handling; Scikit-learn for preprocessing and anomaly models; FastAPI for REST endpoints; PostgreSQL for transactions, predictions and review labels; React for the monitoring dashboard; Docker for repeatable local deployment.

7. High-level system flow

1 INGEST	2 VALIDATE	3 PREPARE	4 SCORE	5 REVIEW
CSV or API transaction	Check schema and required fields	Scale using saved training pipeline	Model returns score and flag	Dashboard displays alert and history

8. Key API & dashboard capabilities

CAPABILITY	PURPOSE
POST /transactions/score	Validate and score one transaction; return transaction ID, anomaly score, flag and model version.
POST /transactions/batch-score	Score an uploaded batch and return a job/result summary.
GET /transactions	List transactions with filters for date, flag, score and pagination.
GET /metrics	Return transaction totals, flagged counts and available evaluation metrics.
Dashboard review	Inspect flagged records, view details, and record analyst disposition (fraud / legitimate / pending).

9. MVP scope

Deliver a Dockerized local application with one documented transaction dataset, schema validation, reproducible preprocessing, training and comparison of all three models, single and batch scoring, PostgreSQL persistence, a basic React review dashboard, and a concise evaluation report. Exclude payment-network integration, automated card blocking, production-grade identity verification, streaming infrastructure, and model retraining automation.

10. Success criteria

MVP is successful when: (1) all three models train and produce repeatable scores from the saved pipeline; (2) valid single and batch requests return a score, flag and model version, while invalid inputs receive clear errors; (3) predictions persist and appear filterably in the dashboard; (4) evaluation reports precision, recall, F1 and PR-AUC on labeled test data, with recall and precision reviewed against a documented threshold; and (5) the full application starts from the provided Docker setup and its API behavior is documented.